

Evaluación de rendimiento de aplicaciones paralelas

Agenda

- Métricas de rendimiento para aplicaciones paralelas. Leyes de Amdahl y Gustafson.
- Metodología experimental para medir el rendimiento de aplicaciones paralelas

Métricas de rendimiento para aplicaciones paralelas

¿Qué tan bien funciona nuestra aplicación
en una cierta máquina paralela?

Métricas:

- Tiempo de ejecución
- Speedup
- Eficiencia paralela
- *Eficiencia energética (no en este curso)*

Escalabilidad: fuerte y débil

Tiempo de ejecución

Tiempo desde el inicio hasta que el último elemento de procesamiento finaliza su ejecución.

Mediciones de tiempo:

- De todo un programa:

```
$ time ./mi_programa
```

- Partes de un programa: instrumentación del código fuente.
 - Función en C: `clock_gettime`
 - OpenMP y MPI agregan funciones propias.

Speedup

- Al evaluar una aplicación paralela, nos interesa conocer cuál es la ganancia de rendimiento (**aceleración**) que se obtiene con el programa paralelo frente a su implementación serie. En este contexto, se define el Speedup como:

$$S_{n,p} = T_{n,1} / T_{n,p}$$

- donde:
 - p es el número de procesadores
 - n es el tamaño del problema
 - $T_{n,1}$ es el tiempo de ejecución del programa serie (en 1 procesador, el mejor algoritmo serie) para tamaño de problema n .
 - T_p es el tiempo de ejecución del programa paralelo con p procesadores idénticos, y un tamaño de problema n .
- El **speedup ideal (o lineal)** se logra cuando $S_{n,p} = p$

Eficiencia Paralela

- La **eficiencia paralela** (en el uso de recursos de cómputo) es una medida que indica la fracción de tiempo en que los recursos están siendo utilizados de forma productiva.

$$\begin{aligned} E_{n, p} &= S_{n, p} / p \\ &= T_{n, 1} / (p * T_{n, p}) \end{aligned}$$

¿Cuál es el máximo valor de E ?

Escalabilidad

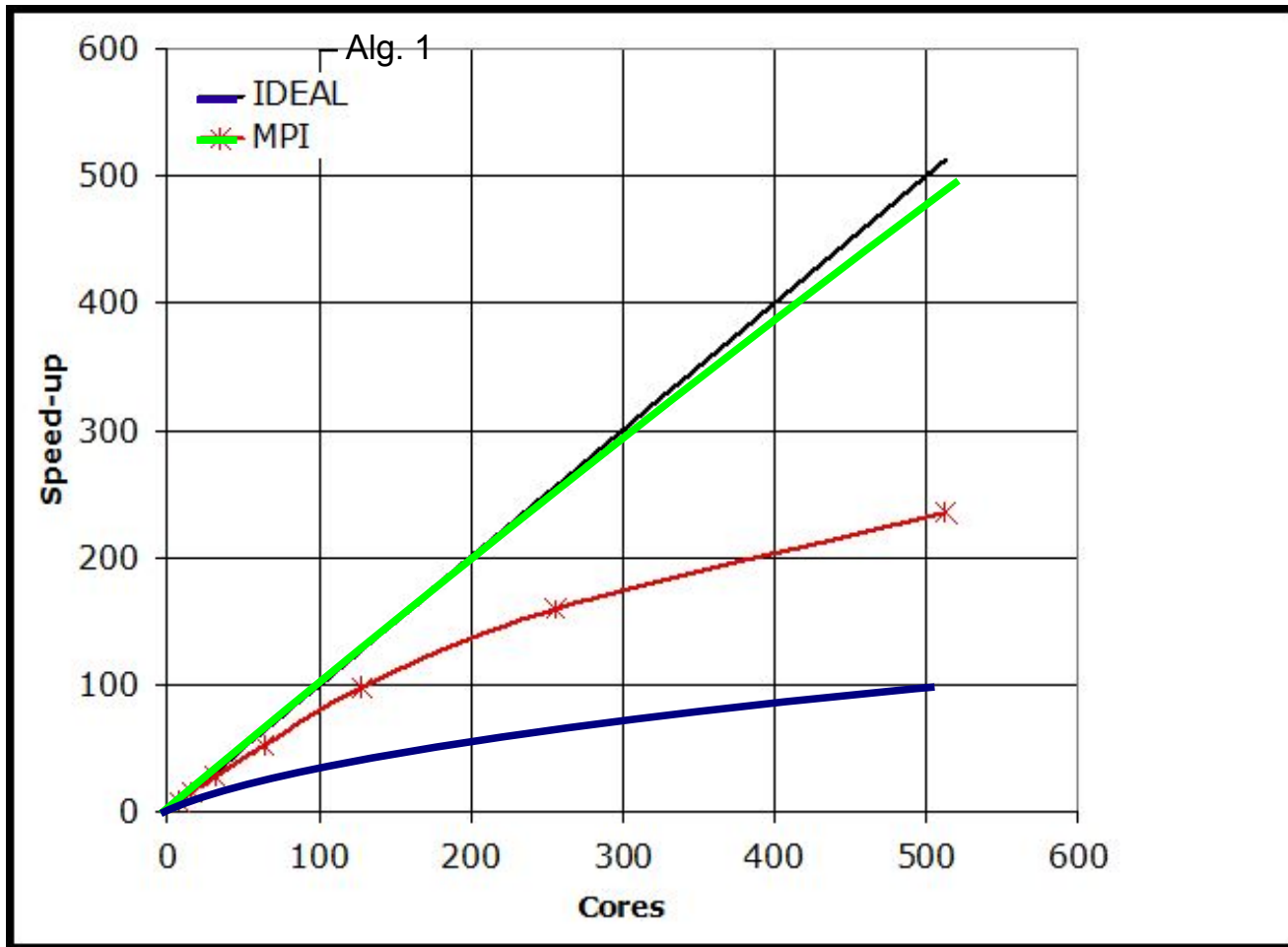
La **escalabilidad** indica cómo cambia el rendimiento de una aplicación paralela cuando aumenta el número de procesadores y posiblemente el tamaño del problema.

Hay dos tipos diferentes de escalabilidad:

- **Escalabilidad fuerte.** Su objetivo es resolver un problema lo más rápido posible:
 - El tamaño del problema permanece fijo, mientras se incrementa el número de procesadores.
- **Escalabilidad débil.** Su objetivo es resolver un problema más grande en igual tiempo:
 - El tamaño del problema se determina de acuerdo al número de procesadores, para mantener una **carga de trabajo fija por procesador**.

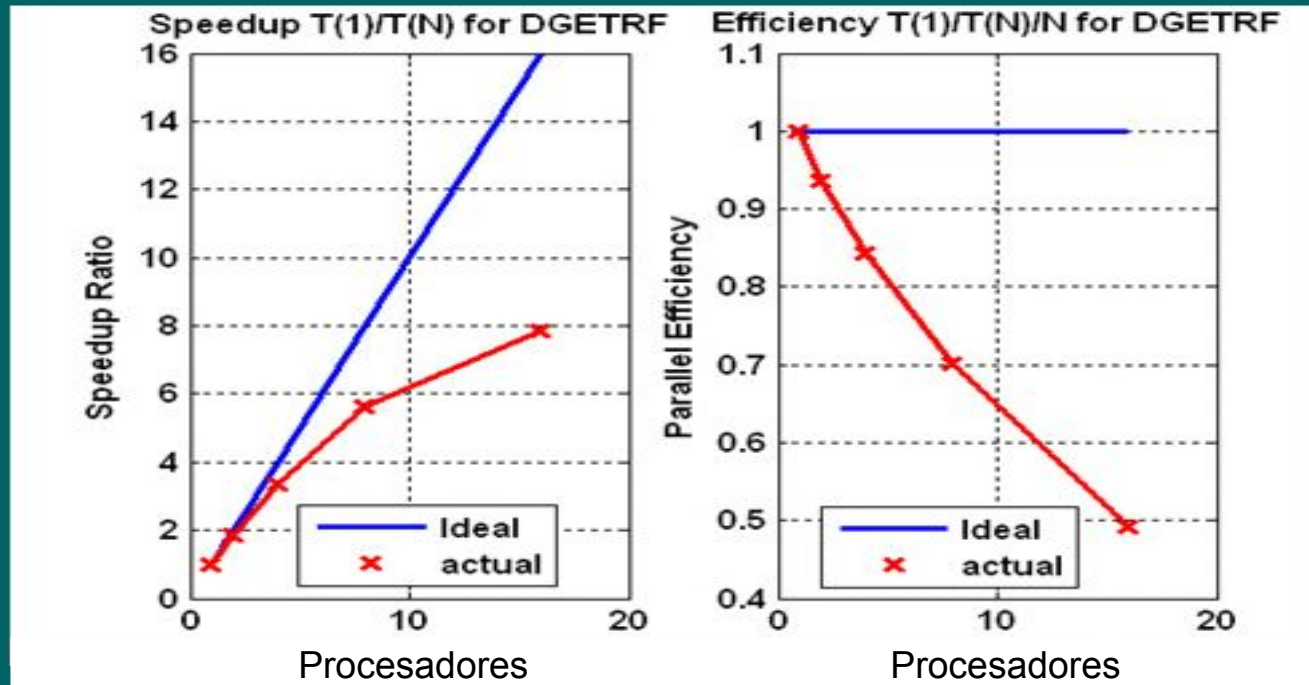
En general, es más difícil lograr la escalabilidad fuerte que la escalabilidad débil.

Escalabilidad fuerte



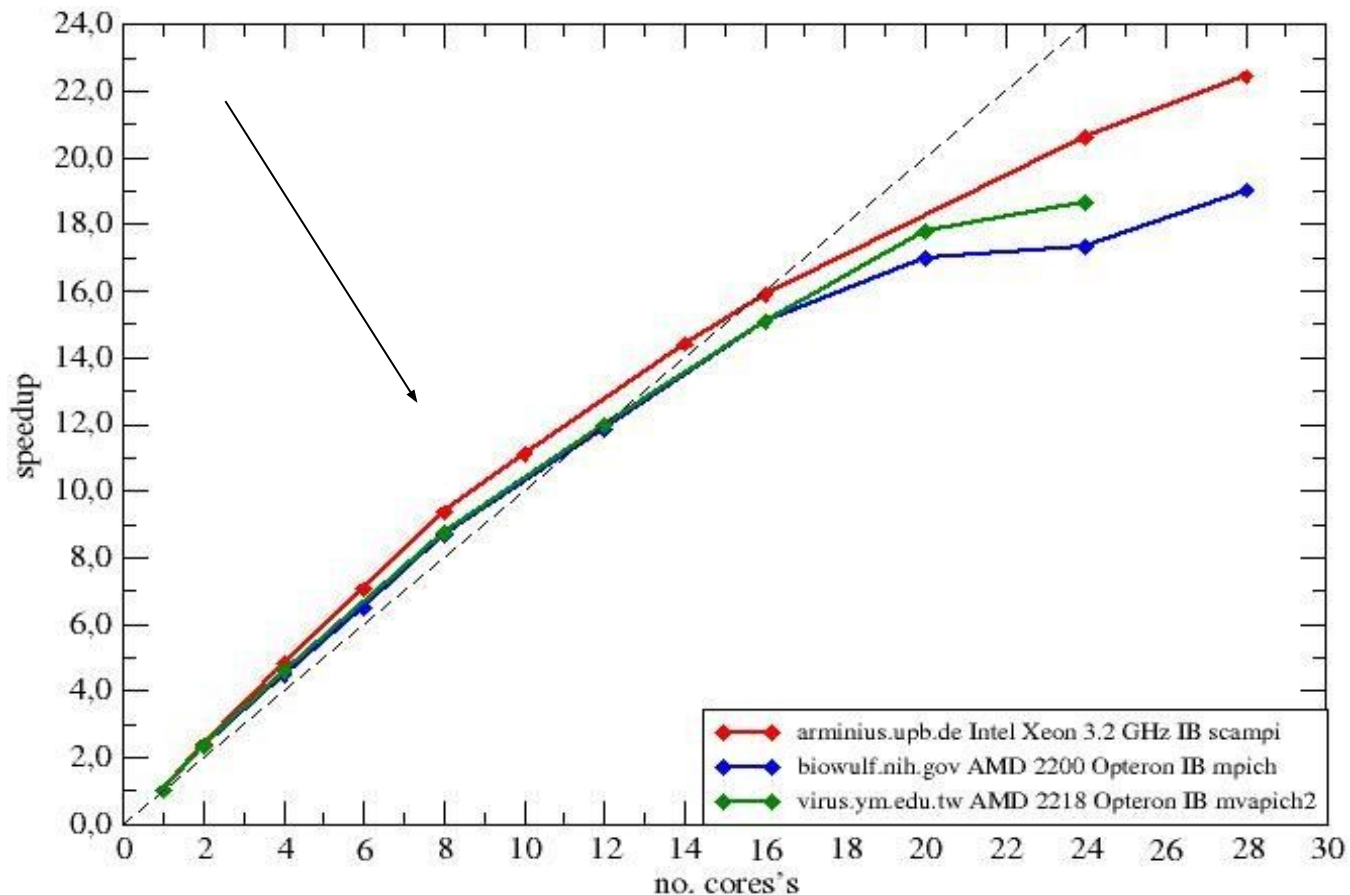
Escalabilidad fuerte

Example of Speedup Ratio & Parallel Efficiency



Escalabilidad fuerte

A veces, se observa un speedup de más de p cuando se usan p procesadores. A este speedup se lo llama **speedup superlineal**.



Escalabilidad fuerte

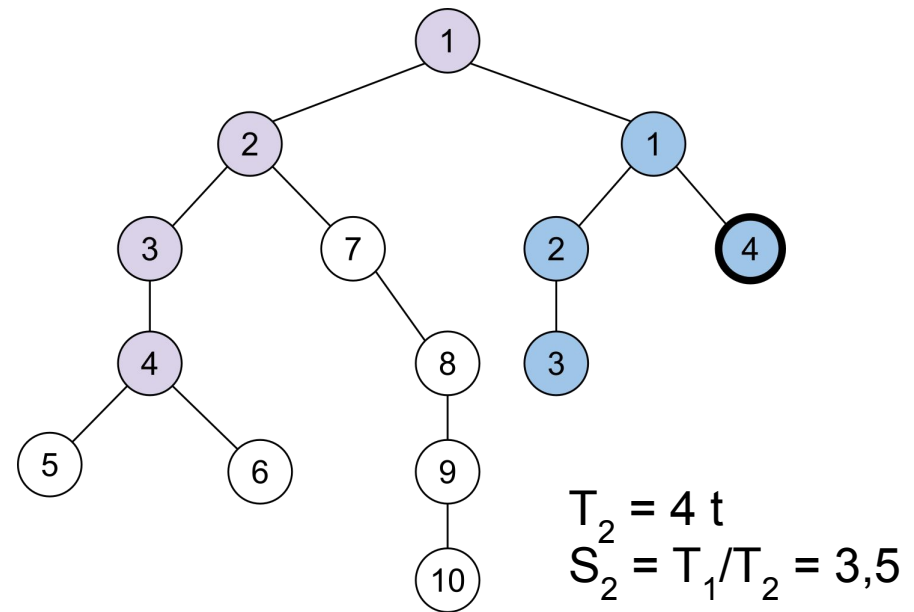
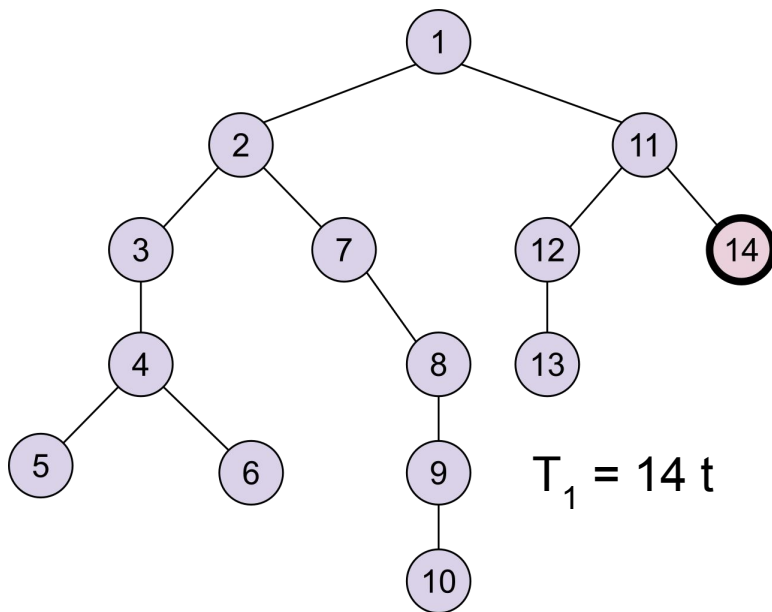
Posibles causas del **speedup superlineal**:

1. Efecto de la memoria: más procesadores (y nodos) implican más memoria de acceso rápido disponible (en orden de velocidad: registros de CPU, L1, L2, L3, RAM, disco local).

Escalabilidad fuerte

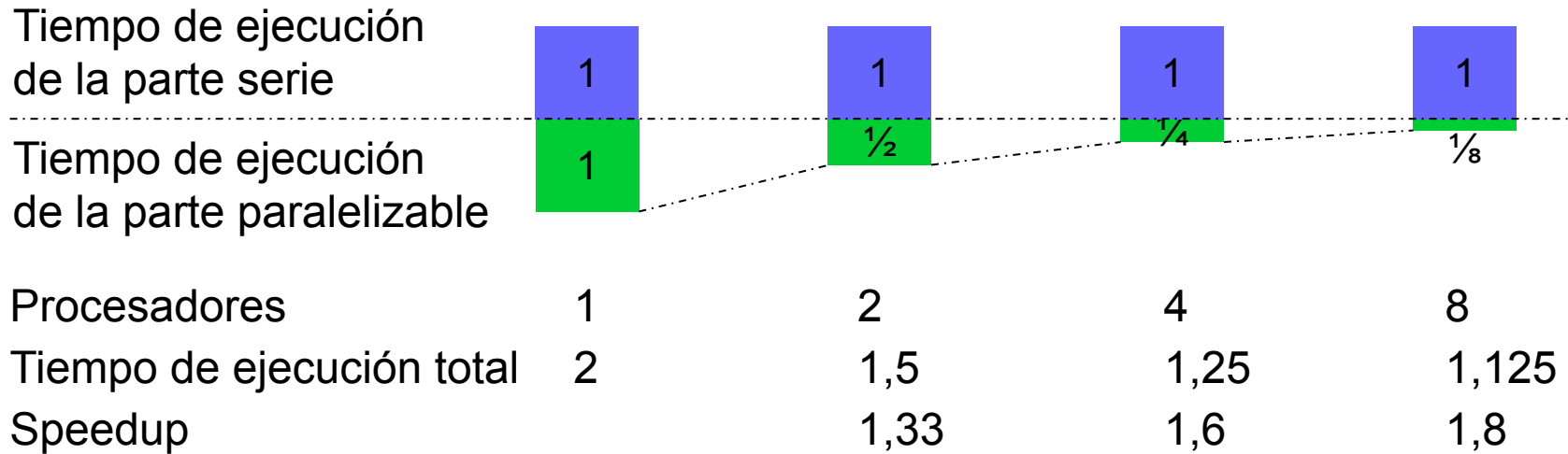
Posibles causas del **speedup superlineal**:

2. Se utiliza *descomposición funcional* en donde el cómputo corresponde a una búsqueda de soluciones (*descomposición exploratoria*). Ejemplo con recorrido *depth-first*:



- Elemento visitado por el procesador 0
- Elemento visitado por el procesador 1
- Elemento buscado

Ley de Amdahl



La mejora del rendimiento ganada por la paralelización está limitada por la proporción del código que es secuencial [Gene Amdahl, 1967].



Ley de Gustafson

En la práctica, **al aumentar el número de CPUs, el problema generalmente se expande** (dimensiones o resolución de una grilla, número de pasos de tiempo, complejidad de cálculos, etc.).

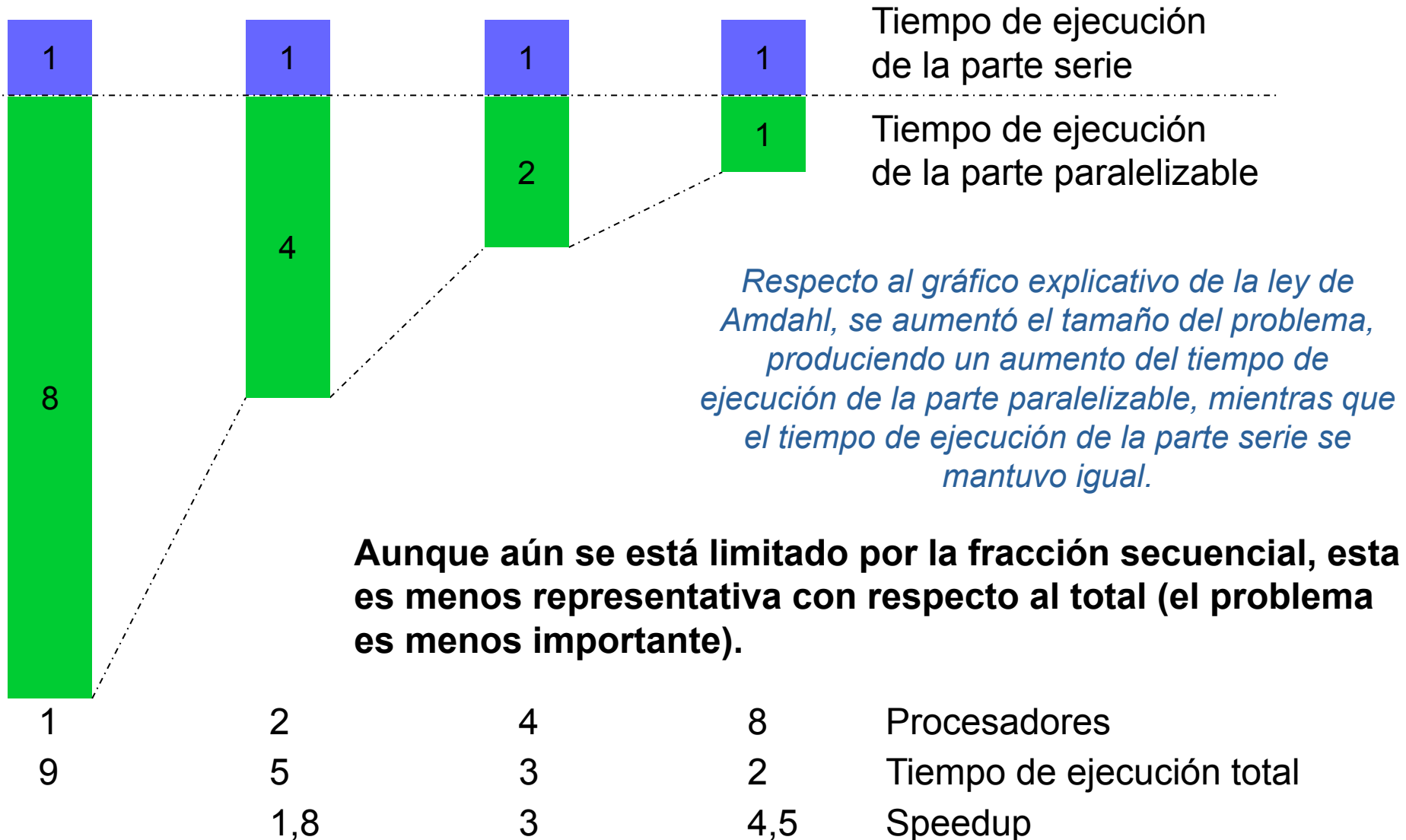
En ciertos casos, la **parte secuencial** del programa tiene un **tiempo de ejecución constante**, independiente del tamaño del problema.

[John Gustafson, 1988]

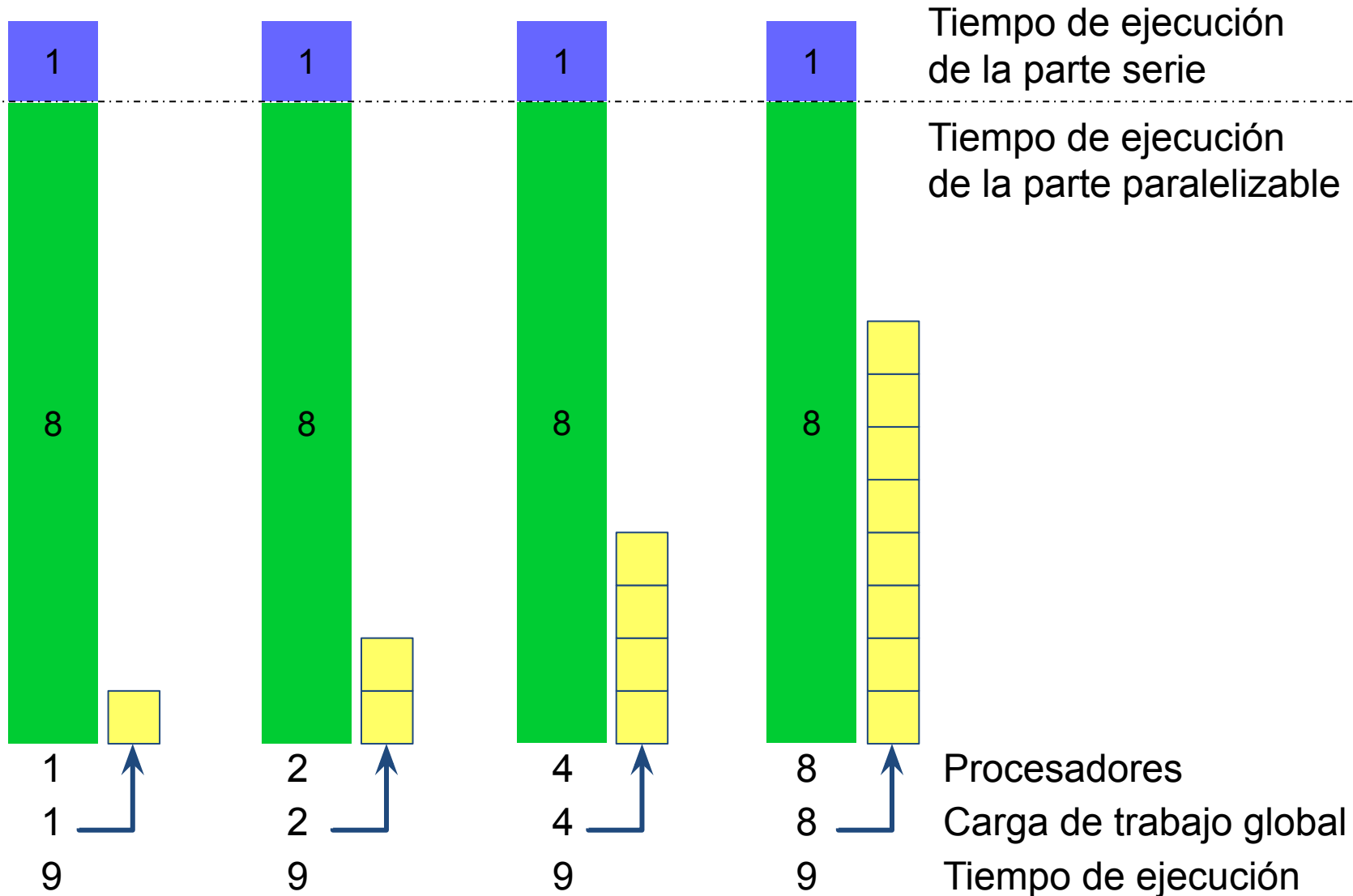
<http://www.johngustafson.net/glaw.html>



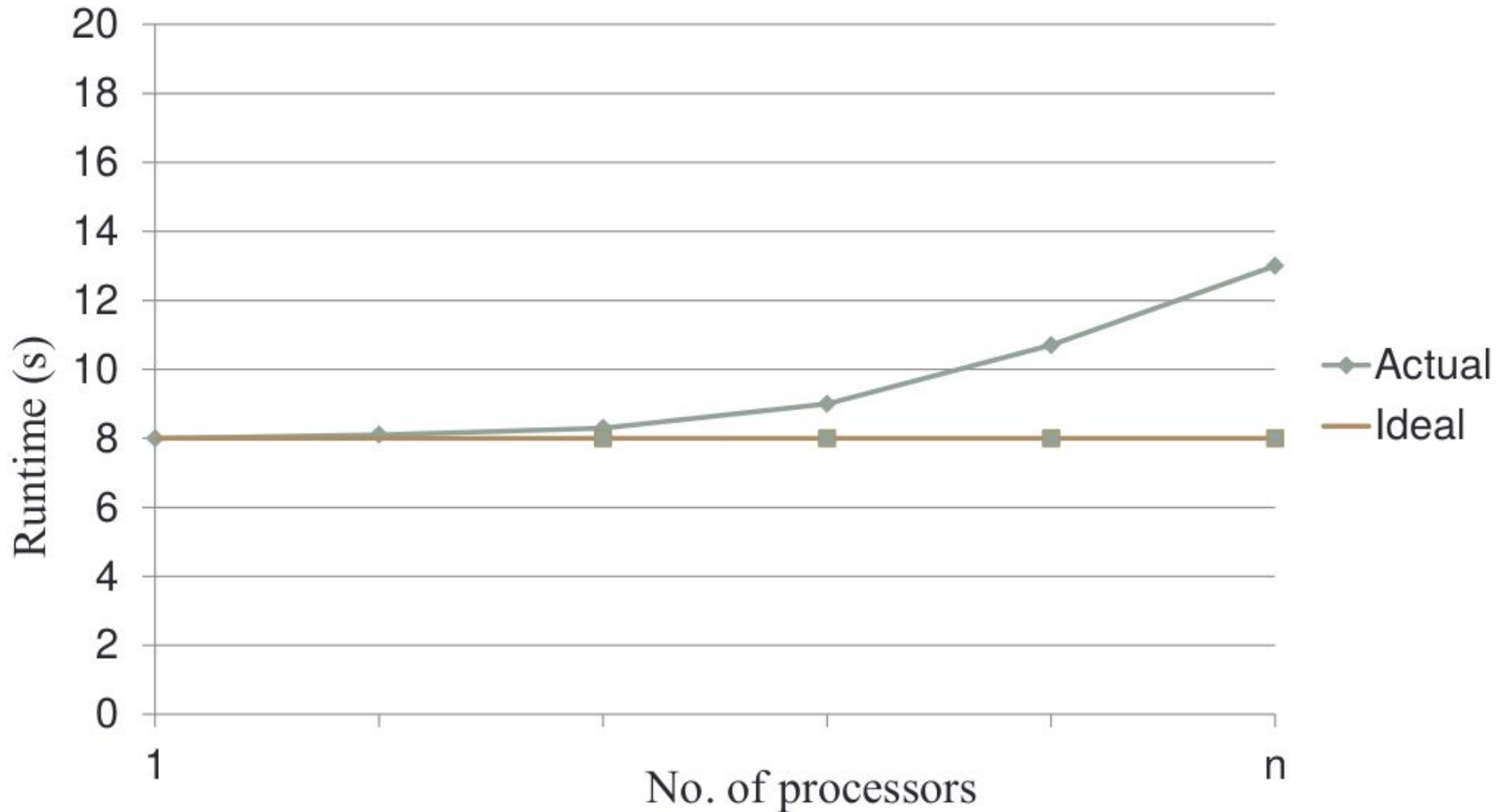
Ley de Gustafson



La ley de Gustafson y la Escalabilidad débil



Escalabilidad débil



Escalabilidad débil



El cálculo del “tamaño” de un problema para producir una determinada “carga de trabajo” puede no ser obvio.

- El tamaño del problema está relacionado a los elementos de datos del problema.
- La carga de trabajo está relacionada a la complejidad temporal del algoritmo que resuelve el problema.

Ejemplo: si un algoritmo tiene una complejidad de $O(N^2)$, duplicar el valor de N produce una carga de trabajo 4 veces mayor. Esto es:
 $(2*N)^2 = 2^2 * N^2 = 4 * N^2$.

¿Cómo se calcula el tamaño del problema
para una carga de trabajo doble?

Escalabilidad débil

Ejemplo de cálculo del tamaño de problema

Suponer que $N_1 = 1000$ es el tamaño de problema utilizado con **1 procesador**.

El cálculo de N_2 es:

$$N_2^2 = 2 * N_1^2$$
$$N_2 = \text{raiz_cuadrada}(2 * N_1^2)$$
$$N_2 = \text{raiz_cuadrada}(2 * 1000^2)$$
$$N_2 = 1414,21$$

El cálculo de N_3 es:

$$N_3^2 = 3 * N_1^2$$
$$N_3 = \text{raiz_cuadrada}(3 * N_1^2)$$
$$N_3 = \text{raiz_cuadrada}(3 * 1000^2)$$
$$N_3 = 1732,05$$

Escalabilidad débil

Un análisis de escalabilidad débil podría incluir un estudio de la **eficiencia paralela**.

Recordemos la fórmula de eficiencia paralela:

$$\begin{aligned} E_{n,p} &= S_{n,p} / p \\ &= T_{n,1} / (p * T_{n,p}) \end{aligned}$$

¿Es esto siempre viable en la práctica?

Ayuda: razonar en relación a los datos necesarios para su cálculo y a la limitación de los recursos físicos

Escalabilidad débil

Como la carga de trabajo total es proporcional al número de procesadores p , podríamos asumir que $T_{n,1} = T_{n1,1} * p$, donde $n1$ es el tamaño de problema utilizado con 1 procesador.

Luego, podemos derivar la siguiente fórmula:

$$\begin{aligned} E_{n,p} &= S_{n,p} / p \\ &= T_{n,1} / (p * T_{n,p}) \\ &= T_{n1,1} * p / (p * T_{n,p}) \\ &= T_{n1,1} / T_{n,p} \end{aligned}$$

Metodología experimental para medir el rendimiento de aplicaciones paralelas

La importancia de realizar mediciones correctas

- Malas mediciones producen una mala evaluación del rendimiento de una aplicación.

Mediciones de tiempo → Métricas de rendimiento → Evaluación de escalabilidad

- Una mala evaluación del rendimiento derivará en decisiones incorrectas, por ejemplo: rediseñar el programa paralelo, ampliar la máquina, etc.

Consideraciones de la experimentación

Normalmente, queremos:

- Tomar mediciones durante **ejecuciones cortas**, y que ellas sean representativas de ejecuciones reales largas. Para esto, es necesario:
 - Construir un modelo de estimación del tiempo de ejecución real.
 - Contemplar:
 - evitar ejecuciones extremadamente cortas
 - el “calentamiento” de la máquina.
- **Mediciones promedio.**
- Mediciones representativas de **ejecuciones típicas.**

Ejecuciones cortas: estimación del tiempo de ejecución total de la aplicación

- Casi toda aplicación tiene como mínimo tres etapas: inicialización, procesamiento central, y finalización.
- En muchas aplicaciones es posible medir una parte de las etapas y luego extrapolar. Por ejemplo, si:

$$\begin{aligned} T_{total} &= T_{inicialización} + T_{procesamiento} + T_{finalización} \\ &= T_{inicialización} + n * T_{subprocesamiento} + T_{finalización} \end{aligned}$$

se podrían medir solo las etapas: $T_{inicialización}$, $T_{subprocesamiento}$, $T_{finalización}$

- Si el impacto de una etapa de inicialización/finalización en la ejecución real es despreciable, en la ejecución corta se podría directamente excluir la etapa (no medirla).

Ejecuciones cortas: calentamiento de la máquina

- El comportamiento de los programas en ejecución es inestable durante un cierto tiempo, y luego se estabiliza.
 - A esta etapa de inestabilidad se la suele denominar “calentamiento”.



- Solución:
 - Sin error: Ejecutar la etapa de calentamiento, durante la cual no se realizan mediciones (o se descartan), para luego comenzar con las mediciones ya en una etapa de estabilidad.
 - Con error tolerable: Realizar ejecuciones lo suficientemente largas como para que la etapa inestable sea despreciable respecto a la etapa estable.

Mediciones promedio

Metodología:

1. Repetir un **mismo experimento**, un número determinado de veces. ¿Cuántas repeticiones? depende de la variabilidad de los resultados observados.
2. Excluir los extremos.
3. Promediar los resultados.

Bibliografía

- Libro de Rauber
4.2 Performance Metrics for Parallel Programs
- Libro de Grama
5.2 Performance Metrics for Parallel Systems
- Ley de Gustafson <http://www.johngustafson.net/glaw.html>